

Day 1: Introduction and Basics

Welcome to Day 1 of the Flutter Workshop. By the end of today, you will understand what Flutter and Dart are, write basic Dart programs, and create your first Flutter app.

Session 1: Understanding Flutter (2 Hours)

1.1 What is Flutter?

Flutter is an open-source UI toolkit by Google for building natively compiled applications for mobile, web, and desktop from a **single codebase**. It uses the **Dart** programming language and provides a rich set of pre-designed widgets.

1.2 What is Dart?

Dart is a client-optimized, object-oriented language developed by Google. It is used to write Flutter apps and can compile to native code (ARM/x86) or JavaScript.

1.3 What is Android Studio / VS Code?

- **Android Studio:** Official IDE for Android development. It includes the Android SDK, emulator, and tools to build, test, and debug Flutter apps.
- **Visual Studio Code (VS Code):** Lightweight, cross-platform code editor. With the Flutter extension, it offers excellent Flutter development support including IntelliSense, debugging, and hot reload.

Your workshop machine already has both tools installed alongside the Flutter SDK.

1.4 Flutter vs Other Frameworks

Feature	Flutter	React Native	Native (Android/iOS)
Language	Dart	JavaScript/TypeScript	Kotlin/Java (Android), Swift/Obj-C (iOS)
Rendering	Own engine (Skia)	Native components via bridge	Native platform widgets
Performance	High (compiles to native code)	Good, but bridge overhead possible	Best
Hot Reload	Yes (sub-second)	Yes (Fast Refresh)	Limited (Instant Run, etc.)
UI Consistency	Pixel-perfect on all platforms	Platform-specific look (or custom)	Native look and feel
Learning Curve	Moderate (Dart + widgets)	Moderate (JS + React)	High (two separate ecosystems)
Single Codebase	Mobile, Web, Desktop	Mobile, Web (React Native Web)	Separate codebases per platform

1.5 Verifying Your Environment (Pre-installed)

Even though everything is set up, run these commands in a terminal to confirm.

Check Flutter SDK

```
flutter --version
```

Expected output: Flutter version number and channel.

Check Dart SDK

```
dart --version
```

Expected output: Dart SDK version (comes bundled with Flutter).

Run Flutter Doctor

```
flutter doctor
```

This checks for any missing dependencies. Your output should show all green checkmarks for:

- Flutter SDK
- Android toolchain
- Android Studio
- VS Code
- Connected device (or emulator)

Sample flutter doctor output:

```
Doctor summary (to see all details, run flutter doctor -v):  
[✓] Flutter (Channel stable, 3.x.x, ...)  
[✓] Android toolchain - develop for Android devices (Android SDK version ...)  
[✓] Chrome - develop for the web  
[✓] Android Studio (version ...)  
[✓] VS Code (version ...)  
[✓] Connected device (1 available)
```

Check Android Studio / Emulator

Open Android Studio, go to **More Actions > AVD Manager** and verify at least one emulator is created (e.g., Pixel 6, API 33). If not, create one quickly.

Session 2: Dart Programming Essentials (2 Hours)

2.0 How to Create and Run Dart Programs

Before diving into Dart syntax, let's learn how to write and execute Dart code.

File Extension

Dart source files use the **.dart** extension.

Creating a Program

1. Open any text editor (VS Code, Notepad, etc.).
2. Create a new file, e.g., `my_program.dart`.
3. Write your Dart code inside.
4. Save the file.

Running a Dart Program

Open a terminal (Command Prompt or PowerShell on Windows) in the directory containing your file and run:

```
dart my_program.dart
```

The program will execute, and any `print()` output will appear in the terminal.

Tip: In VS Code, you can open the file and press `Ctrl+F5` to run without debugging (if the Dart extension is installed).

Now, for each core language feature, we'll first look at the **syntax and its explanation**, then apply it in example programs.

2.1 Variables and Data Types

Syntax & Explanation

```
// Variable declaration
[type] variableName = value;
var variableName = value;      // type inferred
final variableName = value;    // set once at runtime
const variableName = value;    // compile-time constant
dynamic variableName = value;  // can change type later
```

- **type:** one of `int`, `double`, `String`, `bool`, `List`, `Map`, etc.
- **var:** Dart automatically determines the type from the assigned value. Once set, the variable's type is fixed.
- **final:** the value is assigned once and cannot change; it can be set at runtime (e.g., `DateTime.now()`).
- **const:** a compile-time constant; the value must be known before the program runs.
- **dynamic:** a variable that can hold any type and can change its type over time.

Example

```
void main() {
  // Explicitly typed
  String name = 'Flutter Workshop';
  int year = 2026;
  double version = 3.29;
```

```

bool isAwesome = true;

print('Name: $name');
print('Year: $year');
print('Version: $version');
print('Is Awesome: $isAwesome');

// Type inference with var
var message = 'Hello Dart'; // inferred as String
var count = 42; // inferred as int

print('Message: $message');
print('Count: $count');

// dynamic (can change type)
dynamic anything = 'text';
print('Anything (initial): $anything');
anything = 100;
print('Anything (changed): $anything');

// final and const
final now = DateTime.now(); // set once at runtime
const pi = 3.1416; // compile-time constant

print('Current Time (final): $now');
print('Pi (const): $pi');
}

```

Data Types Table

Type	Description	Example
int	Integer numbers	10, -3
double	Floating-point numbers	3.14, -0.5
String	Sequence of characters	'hello', "dart"
bool	true/false	true, false
List	Ordered collection	[1, 2, 3]
Map	Key-value pairs	{'name': 'Flutter'}
dynamic	Disable type checking	anything

2.2 Functions

Syntax & Explanation

```

// Basic function
returnType functionName(parameters) {
  // body
  return value;
}

// Arrow syntax (for single expressions)
returnType functionName(parameters) => expression;

// Optional positional parameters (enclosed in [])
returnType functionName(param1, [optionalParam]) { ... }

```

```
// Named parameters with default values (enclosed in {})  
returnType functionName({param1 = defaultValue, required param2}) { ... }
```

- **returnType**: the type of value the function returns (e.g., `int`, `String`, `void` for no return).
- **parameters**: comma-separated list of inputs.
- **Optional positional parameters**: placed inside `[]`; they can be omitted when calling.
- **Named parameters**: placed inside `{}`; callers must use the parameter name, and you can set default values or mark them `required`.

Example

```
// Basic function  
int add(int a, int b) {  
    return a + b;  
}  
  
// Arrow syntax (single expression)  
int multiply(int a, int b) => a * b;  
  
// Optional positional parameters  
String greet(String name, [String? title]) {  
    if (title != null) return 'Hello, $title $name';  
    return 'Hello, $name';  
}  
  
// Named parameters with default values  
void describe({String name = 'Unknown', int age = 0}) {  
    print('$name is $age years old');  
}  
  
void main() {  
    print(add(2, 3));           // 5  
    print(greet('Alice'));     // Hello, Alice  
    describe(name: 'Bob', age: 25); // Bob is 25 years old  
}
```

2.3 Control Structures

Syntax & Explanation

if-else

```
if (condition) {  
    // executed when condition is true  
} else if (anotherCondition) {  
    // executed when anotherCondition is true  
} else {  
    // executed when all previous conditions are false  
}
```

for loop

```
for (initialization; condition; update) {  
    // body  
}
```

```
}
```

for-in loop (iterate over elements of a collection)

```
for (var item in collection) {  
    // use item  
}
```

while loop

```
while (condition) {  
    // body runs while condition is true  
}
```

switch

```
switch (variable) {  
    case value1:  
        // code  
        break;  
    case value2:  
        // code  
        break;  
    default:  
        // fallback  
}
```

Example

```
void main() {  
    // if-else  
    int score = 85;  
    if (score >= 90) {  
        print('A');  
    } else if (score >= 80) {  
        print('B');  
    } else {  
        print('C');  
    }  
  
    // for loop  
    for (int i = 0; i < 3; i++) {  
        print('Count: $i');  
    }  
  
    // for-in loop  
    var items = ['Dart', 'Flutter', 'Widgets'];  
    for (var item in items) {  
        print(item);  
    }  
  
    // while loop  
    int x = 0;  
    while (x < 3) {  
        print(x);  
        x++;  
    }  
  
    // switch  
    var command = 'START';  
    switch (command) {  
        case 'START':
```

```

        print('Starting...');
        break;
    case 'STOP':
        print('Stopping...');
        break;
    default:
        print('Unknown');
    }
}

```

2.4 Collections

Syntax & Explanation

List – ordered, indexed collection.

```

List<type> listName = [element1, element2];
// OR
var listName = <type>[element1, element2];

```

Set – unordered collection of unique elements.

```

Set<type> setName = {element1, element2};

```

Map – key-value pairs.

```

Map<keyType, valueType> mapName = {
    key1: value1,
    key2: value2,
};

```

Spread operator (...) – inserts all elements of another collection into a new collection literal.

Example

```

void main() {
    // List
    List<String> fruits = ['Apple', 'Banana', 'Cherry'];
    fruits.add('Date');
    fruits.removeAt(0);
    print(fruits.length); // 3
    print(fruits[1]);      // Cherry

    // Set (unique elements)
    Set<int> numbers = {1, 2, 3, 2};
    print(numbers); // {1, 2, 3}

    // Map
    Map<String, dynamic> user = {
        'name': 'Alice',
        'age': 30,
        'isAdmin': false,
    };
    print(user['name']);
    user['age'] = 31;

    // Spread operator (...)
    var moreFruits = ['Mango', ...fruits];
}

```

2.5 Object-Oriented Programming (OOP)

Syntax & Explanation

Class definition

```
class ClassName {
    // fields (properties)
    type fieldName;

    // constructor
    ClassName(this.fieldName, ...); // shorthand for assigning parameters

    // named constructor
    ClassName.namedConstructor(...) : field = value;

    // methods
    returnType methodName() {
        // body
    }
}
```

Inheritance

```
class ChildClass extends ParentClass {
    ChildClass(params) : super(parentParams); // calls parent constructor

    @override
    returnType methodName() {
        // new implementation
    }
}
```

- `extends` is used to create a subclass.
- `super` refers to the parent class.
- `@override` annotation marks an overridden method (optional but recommended).

Example

```
class Animal {
    String name;
    int age;

    // Constructor
    Animal(this.name, this.age);

    // Named constructor
    Animal.baby(this.name) : age = 0;

    // Method
    void speak() {
        print('$name makes a sound.');
```

```
}

// Inheritance
class Dog extends Animal {
    String breed;
```

```

Dog(String name, int age, this.breed) : super(name, age);

@override
void speak() {
  print('$name barks!');
}
}

class Cat extends Animal {
  Cat(String name, int age) : super(name, age);

  @override
  void speak() {
    print('$name meows.');
  }
}

void main() {
  var dog = Dog('Rex', 5, 'Labrador');
  dog.speak(); // Rex barks!

  var cat = Cat('Whiskers', 3);
  cat.speak(); // Whiskers meows.

  // Using a base class reference
  List<Animal> zoo = [dog, cat];
  for (var animal in zoo) {
    animal.speak();
  }
}

```

Session 3: First Flutter App (2 Hours)

3.1 Understanding Flutter Widgets

Everything in Flutter is a **widget**. The UI is built by composing widgets in a tree structure. Two primary widget types:

- **StatelessWidget**: Immutable, no internal state.
- **StatefulWidget**: Mutable state, can rebuild when state changes.

Widget Tree Example (ASCII Line Image)

```

MaterialApp
├── Scaffold
│   ├── AppBar
│   │   └── Text("My App")
│   └── Center
│       └── Text("Hello, World!")

```

3.2 Creating a New Flutter Project

Open a terminal and navigate to your workspace, then run:

```
flutter create hello_world
```

This creates a folder named `hello_world` with all necessary files.

3.3 The Default Counter App Structure

Explore `lib/main.dart`. You'll see a `StatefulWidget` with a counter. We'll replace it with our own Hello World.

3.4 Hello World App (Stateless)

Replace the entire contents of `lib/main.dart` with:

```
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Hello World',
      theme: ThemeData(primarySwatch: Colors.blue),
      home: Scaffold(
        appBar: AppBar(
          title: const Text('My First Flutter App'),
        ),
        body: const Center(
          child: Text(
            'Hello, World!',
            style: TextStyle(fontSize: 24, color: Colors.deepPurple),
          ),
        ),
      ),
    );
  }
}
```

3.5 Running the App

Make sure an emulator is running or a device is connected. From the project directory:

```
cd hello_world
flutter run
```

You will see the app launch. The terminal will show a message like:

```
Flutter run key commands.
r Hot reload. ☐☐☐
R Hot restart.
h List all available interactive commands.
d Detach (terminate "flutter run" but leave app running).
c Clear the screen
q Quit (terminate the application on the device).
```

3.6 Hot Reload and Hot Restart

- **Hot Reload (r):** Injects updated source code into the running Dart VM without restarting

the app or losing state. Very fast.

- **Hot Restart (R):** Resets the app state and rebuilds the widget tree from scratch.

Try changing the text style or the message in your editor, save the file, then press `r` in the terminal. The change should appear instantly.

3.7 Debugging Basics

- **Debug Console:** `print()` statements appear in the terminal/console.
- **Debugger:** Set breakpoints in VS Code (click left of line number) or Android Studio. When the app hits the breakpoint, you can inspect variables, step through code.
- **Flutter Inspector:** In VS Code, open the Command Palette (`Ctrl+Shift+P`) and search "Flutter: Open DevTools" to access the widget inspector, timeline, and more.

Common Debugging Commands

```
print('Debug value: ${someVariable}');  
debugPrint('Long text that might be truncated by print');
```

Day 1 Wrap-Up

Today you learned:

- What Flutter, Dart, and the development tools are.
- How Flutter compares to other frameworks.
- How to create and run `.dart` files, and the core Dart language syntax: variables, functions, control flow, collections, and OOP.
- Creating and running a basic Flutter app with Hot Reload.

Hands-on Practice:

1. Create a new Dart file `practice.dart` and write a function that takes a list of numbers and returns their sum. Run it with `dart practice.dart`.
2. Modify the Hello World app to display your name in the center.
3. Add a `Text` widget below the greeting displaying the current time (hint: `DateTime.now()`).
4. Change the `AppBar` title and theme color.

Tomorrow: You'll dive into UI development with layouts, user input, navigation, and state management.

All commands verified for Windows 10/11 with pre-installed Flutter SDK, Android Studio, and VS Code.